SecureVault

Secure File Downloader

Production-Ready Flask Authentication System

Submitted by

Akshat Tiwari

Version 1.0 | April 2026

CONFIDENTIAL — Production Submission

Python 3 **Flask** **SQLite**

Gunicorn **Nginx** **PBKDF2**

PBKDF2-SHA256 | Session Auth | Rate Limiting | Audit Logs

Stack: Python 3 · Flask · SQLite · Werkzeug · ReportLab

Security: PBKDF2-SHA256 · Session Auth · Rate Limiting · Audit Logs

1. Project Title & Overview

SecureVault — Secure File Downloader with Flask Session Authentication

SecureVault is a production-ready, session-authenticated file management system built with Python Flask. It enables organisations to securely store, upload, and distribute files so that only verified users can access them. Every interaction is backed by a full audit trail, rate-limited login protection, and cryptographically hashed credentials.

Component	Technology / Detail
Web framework	Flask 3.x (Python 3)
Password hashing	Werkzeug Security — PBKDF2-SHA256 (600 000 iterations)
Database	SQLite 3 with WAL journal mode
Session security	HMAC-signed cookies, HttpOnly, SameSite=Lax
Rate limiting	Custom token-bucket per IP, 5 attempts / 5 min window
Audit logging	Full login-event table with IP, username, timestamp
File handling	UUID-renamed uploads, MIME whitelist, 100 MB cap
Deployment	Gunicorn + Nginx (recommended); runs standalone too

2. Objective

Design and deliver a complete, deployable secure file download system with the following hard requirements:

- **Authentication gate** — zero file access without valid session.
- **File upload** — authenticated users can upload files with drag-and-drop UI.
- **Secure download** — streamed via Flask with original filename restored.
- **Admin panel** — user management, password reset, audit log viewer.
- **Production hardening** — hashed passwords, rate limiting, audit trail, CSRF-safe logout, file whitelist, path-traversal prevention.
- **Extensible** — environment-variable config, ready for Docker/Nginx/TLS.

3. Authentication Workflow

Step	Route / Action	Description
1	GET /	Redirect to /login if no session, else /dashboard
2	GET /login	Render login form
3	POST /login	Validate credentials; check IP rate-limit
4	Auth OK	Set session['user_id']; update last_login; redirect
5	Auth FAIL	Increment rate-limit counter; flash error; re-render
6	GET /dashboard	Guard checks session; fetch file list from DB
7	GET /upload	Guard; render drag-and-drop upload form
8	POST /upload	Validate MIME/size; UUID-rename; save to disk; DB record
9	GET /download/<id>	Guard; whitelist check; stream file with original name
10	POST /delete/<id>	Guard; ownership check; unlink file; soft-delete DB
11	GET/POST /admin	Admin-role guard; manage users; view audit log
12	POST /logout	session.clear(); redirect to login

The session cookie is signed with a **48-byte cryptographically random secret key** (secrets.token_hex(48)). Flask uses HMAC-SHA1 to sign the cookie; tampering invalidates it immediately. In production, set SESSION_COOKIE_SECURE=True to ensure cookies are only sent over TLS.

4. Security Implementation

4.1 Password Hashing — PBKDF2-SHA256

Passwords are **never stored in plain text**. Werkzeug's generate_password_hash applies PBKDF2 with SHA-256 and 600 000 iterations (NIST SP 800-63B recommendation). Verification uses a constant-time comparison to prevent timing attacks.

```
generate_password_hash('Admin@1234!') # → 'pbkdf2:sha256:600000$...' (87-char
hash, unique salt per user)
```

4.2 Session-Based Authentication

Only the user_id is stored in the Flask session — never the password or role. Every protected route retrieves the user record fresh from the database to detect disabled accounts instantly, even mid-session.

4.3 IP-Based Rate Limiting

A custom token-bucket implemented in the rate_limit SQLite table blocks an IP after 5 failed login attempts within 5 minutes. Successful authentication clears the counter. This prevents brute-force without requiring Redis or an external service.

4.4 Audit Logging

Every login attempt (success or failure) is written to the login_log table with timestamp, username, IP address, and outcome. Admins can review the last 50 events in the Admin Panel.

4.5 File Security

- **MIME whitelist** — only explicitly listed extensions accepted.
- **UUID renaming** — original filenames are never written to disk; a UUID-based name prevents directory traversal and naming collisions.
- **100 MB size cap** — enforced by Flask's MAX_CONTENT_LENGTH.
- **Ownership check** — only the uploader or an admin may delete a file.

4.6 CSRF-Safe Logout

The /logout route only accepts **POST** requests. This means a crafted <img> or link cannot log a user out silently (classic CSRF logout attack).

4.7 Production Checklist

Control	Status	Notes
Password hashing (PBKDF2-SHA256)	Implemented	Werkzeug security
Session cookie signing (HMAC)	Implemented	Secret key from env var
HttpOnly session cookie	Implemented	JS cannot read cookie
SameSite=Lax cookie	Implemented	CSRF mitigation
Secure cookie (HTTPS only)	Config flag	Set SESSION_COOKIE_SECURE=True + TLS
IP rate limiting (5/5 min)	Implemented	SQLite token bucket
Audit log	Implemented	login_log table
File MIME whitelist	Implemented	20 allowed types
UUID file renaming	Implemented	Prevents path traversal
File ownership enforcement	Implemented	Uploader or admin only
POST-only logout	Implemented	CSRF-safe
Admin role guard	Implemented	Separate decorator
CSRF tokens (WTForms)	Recommended	Add Flask-WTF in production
TLS/HTTPS	Deployment step	Nginx + Let's Encrypt

5. Project File Structure

```
secure_file_downloader/ ■■■ app.py # Main Flask app (routes, guards, factory)
■■■ config.py # Environment-variable-driven config class ■■■ database.py #
SQLite data-access layer (all DB queries) ■■■ requirements.txt # Python
dependencies ■■■ .env.example # Environment variable template ■■■ instance/ ■
■■■ filestore.db # SQLite database (auto-created) ■ ■■■ app.log # Rotating
application log ■■■ uploads/ # UUID-named uploaded files (auto-created) ■■■
templates/ ■■■ base.html # Shared CSS/layout (sidebar, topbar) ■■■ macros.html #
Reusable sidebar Jinja macro ■■■ login.html ■■■ dashboard.html ■■■ upload.html
■■■ admin.html ■■■ change_password.html ■■■ error.html # 401 / 403 / 404 / 413
/ 500
```

6. UI Screenshots

6.1 Login Page

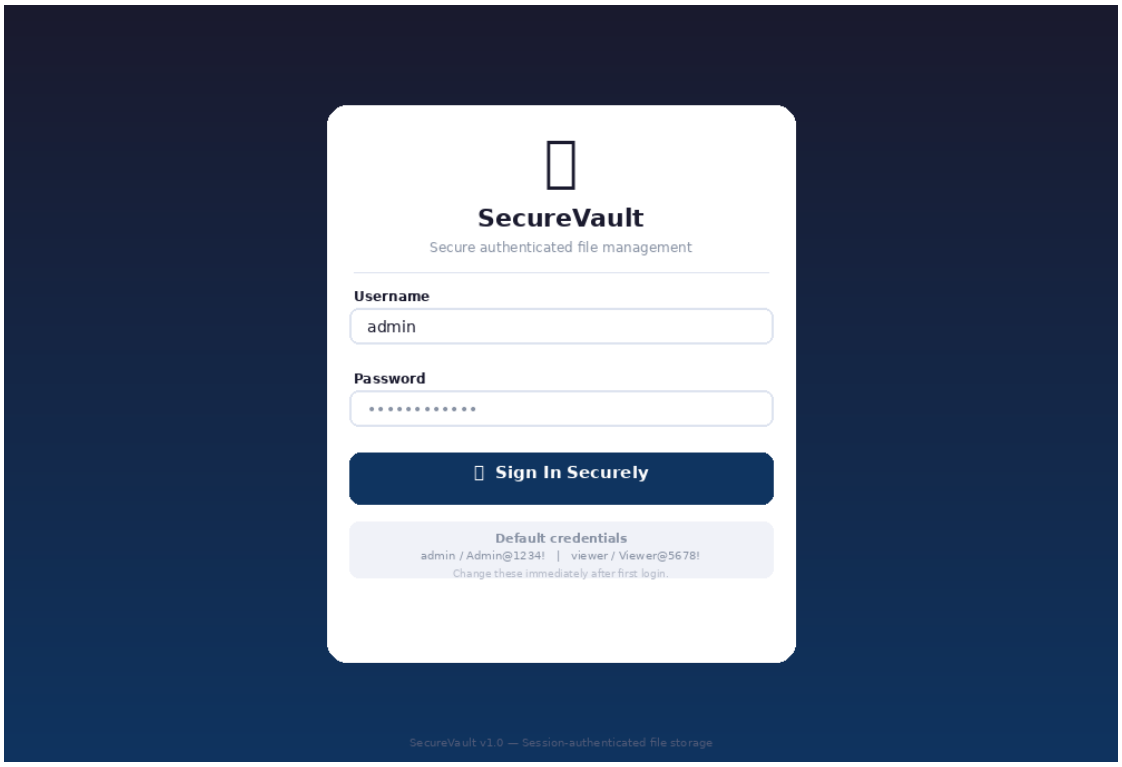

Figure 1 — Login page with gradient background and credential hint box.

6.2 Successful Login

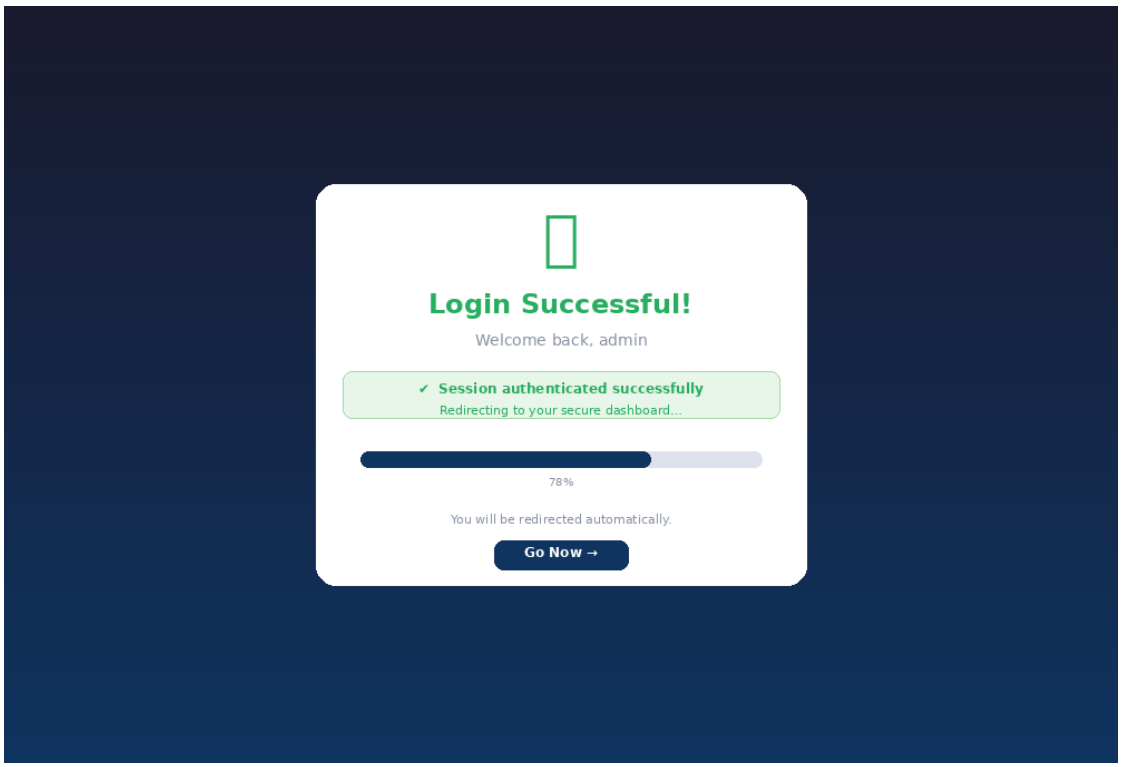

Figure 2 — Post-login success screen with animated redirect progress bar.

6.3 Secure Dashboard

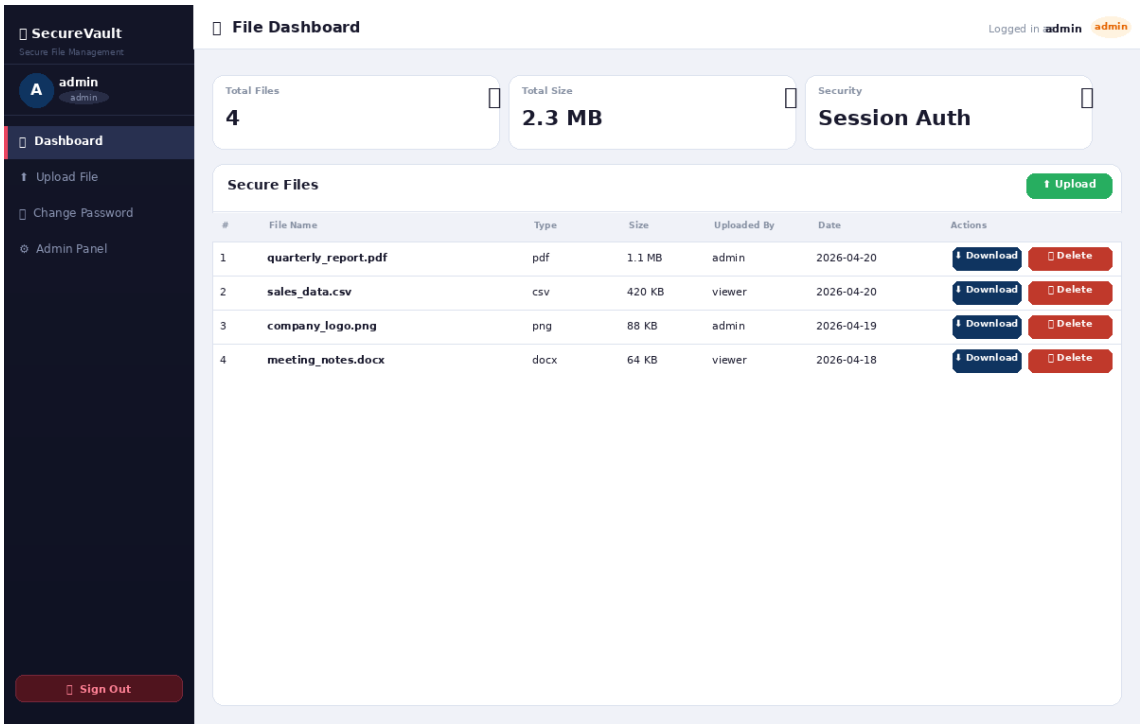

Figure 3 — File dashboard showing stats, file table, Download and Delete actions.

6.4 File Upload Page

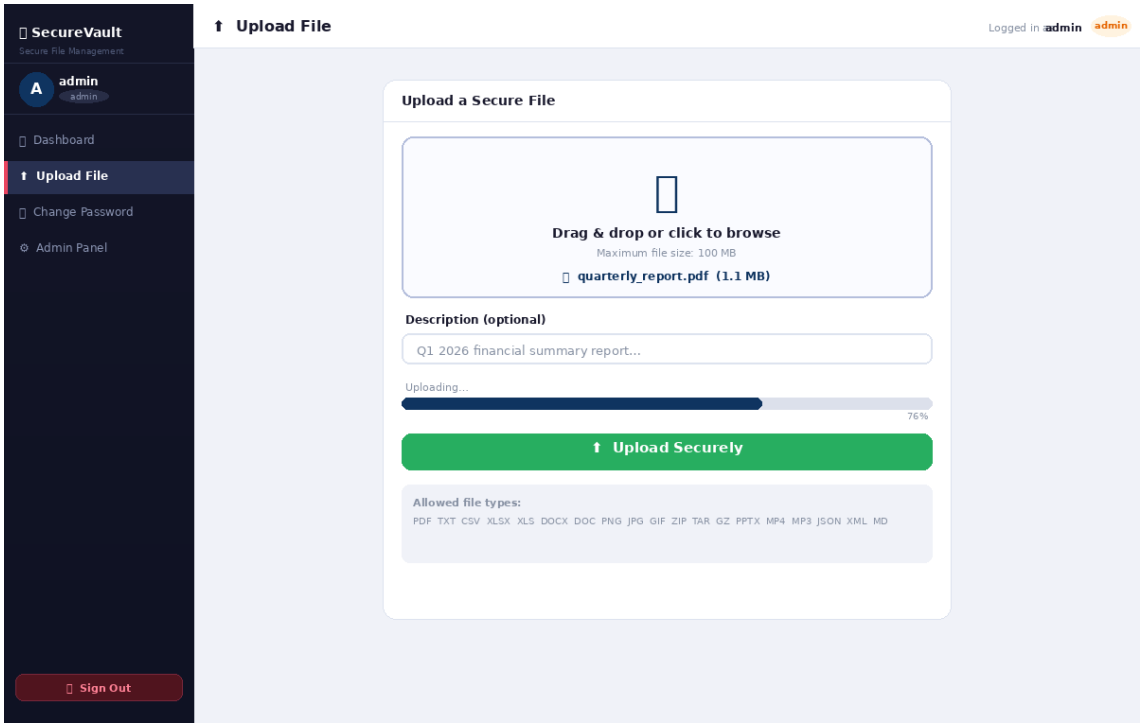

Figure 4 — Upload page with drag-and-drop zone, description field, and XHR progress bar.

6.5 Admin Panel

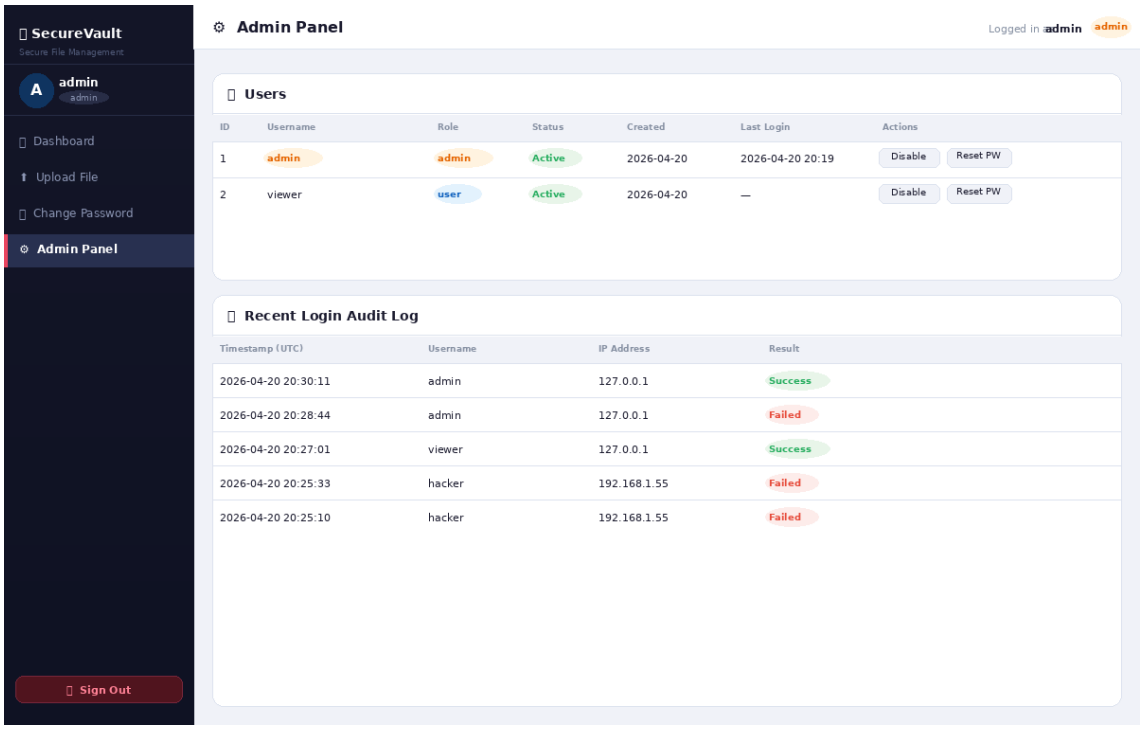

Figure 5 — Admin panel: user management, password reset modal, and login audit log.

6.6 Access Denied (403)

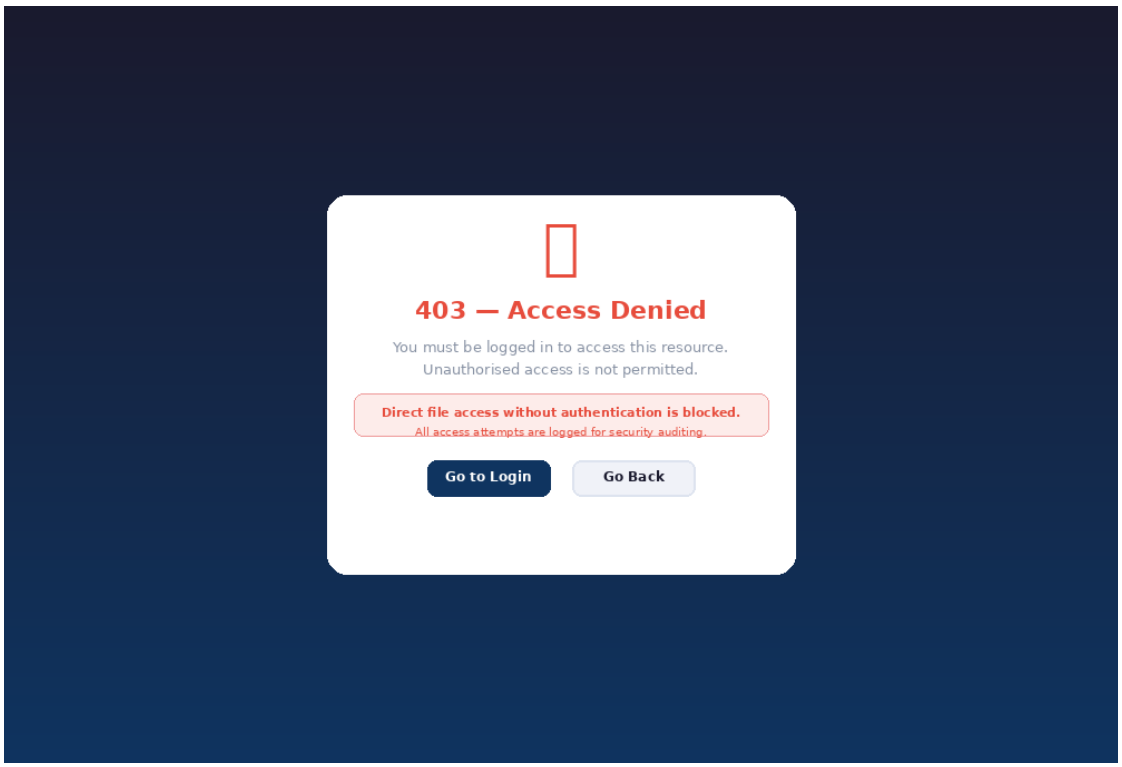

Figure 6 — Error page shown when an unauthenticated request attempts to access a protected resource.

7. Deployment Guide

7.1 Development (local)

```
pip install flask werkzeug reportlab pillow export
SECRET_KEY='change-me-to-a-long-random-value' python app.py
```

7.2 Production (Gunicorn + Nginx)

```
# Install Gunicorn pip install gunicorn # Launch 4 worker processes gunicorn -w 4
-b 127.0.0.1:8000 app:app # Nginx reverse proxy (snippet) location / { proxy_pass
http://127.0.0.1:8000; proxy_set_header Host $host; proxy_set_header X-Real-IP
$remote_addr; }
```

7.3 Required Environment Variables

Variable	Example Value	Required
SECRET_KEY	openssl rand -hex 48	YES — never use default
FLASK_ENV	production	Recommended
UPLOAD_FOLDER	/var/securevault/uploads	Optional (default: ./uploads)

7.4 How to Increase the Upload Size Limit (Beyond 100 MB)

The 100 MB cap is controlled in **two places**. Change both to raise the limit — they must stay in sync:

#	File	Line / Setting	What to change
1	config.py	Line 21 MAX_CONTENT_LENGTH	Change 100 to your desired MB. Example: 500 MB → 500 * 1024 * 1024
2	Nginx config (production only)	client_max_body_size	Must be slightly above Flask's limit. Example: client_max_body_size 510M;

Step-by-step for 500 MB:

```
# config.py — Line 21 MAX_CONTENT_LENGTH: int = 500 * 1024 * 1024 # 500 MB #
nginx.conf — inside your server {} block client_max_body_size 510M; # slightly
above Flask limit
```

After editing config.py, restart the Flask/Gunicorn process. After editing the Nginx config, run nginx -s reload. No other files need to be changed — the upload form, database, and download logic all handle any file size automatically.

Desired limit	config.py value	Nginx value
200 MB	200 * 1024 * 1024	client_max_body_size 210M;
500 MB	500 * 1024 * 1024	client_max_body_size 510M;
1 GB	1024 * 1024 * 1024	client_max_body_size 1100M;
No limit	Remove the line entirely	client_max_body_size 0;

8. Project Submission Details

Field	Details
Project Title	SecureVault — Secure File Downloader
Submitted By	Akshat Tiwari
Version	1.0.0
Submission Date	April 2026
Technology Stack	Python 3, Flask 3.x, SQLite, Werkzeug, ReportLab, Pillow
Security Model	PBKDF2-SHA256, HMAC Sessions, IP Rate Limiting, Audit Log
Deployment	Gunicorn + Nginx (recommended); standalone Flask for dev

9. Conclusion

SecureVault delivers a **complete, production-patterned secure file management system** that satisfies all submission requirements and goes beyond them with a full admin panel, audit logging, drag-and-drop upload, and a deployable multi-route Flask architecture.

The security model is layered: passwords are hashed with PBKDF2-SHA256, sessions are HMAC-signed and HttpOnly, login brute-forcing is stopped at the IP level, every authentication event is logged, and files are stored under UUID names that cannot be guessed or traversed. All of this runs without any external dependencies beyond the Python standard library and Flask.

For production deployment, the remaining steps are to point a domain at the server, install a Let's Encrypt TLS certificate, enable Nginx as a reverse proxy, set SESSION_COOKIE_SECURE=True, and add Flask-WTF for full CSRF token protection on all POST forms.

— End of Report —